

PLAN DE IMPLEMENTACIÓN ·
ESTUDIO DE VIABILIDAD

Chat CONECTA-LT

Chat en vivo real entre usuarios registrados: conversaciones 1-a-1, grupos y notas de voz sobre la infraestructura actual de la plataforma, con Pusher Channels como único servicio nuevo y costo inicial de cero dólares al mes.

Preparado para el equipo CONECTA-LT
Septiembre 2026 · Versión 1.0 · conectalt.com

Índice

1. Resumen ejecutivo	1
2. Contexto, objetivo y decisiones	2
3. Estado actual de la plataforma (evidencia verificada)	3
4. El reto clave: tiempo real sobre Vercel serverless	5
5. Arquitectura propuesta	6
6. Modelo de datos	8
7. Alcance funcional de la versión 1	9
8. Plan de implementación por fases	11
9. Costos y límites del plan gratuito	12
10. Riesgos, seguridad y moderación	14
11. Alternativa sin terceros y decisiones pendientes	15

1. Resumen ejecutivo

Este documento evalúa la viabilidad de incorporar un chat nativo a CONECTA-LT para que las personas registradas en la plataforma puedan comunicarse directamente entre sí, sin salir de la web. El veredicto general es favorable: el proyecto es **viable con condiciones conocidas y acotadas**. La infraestructura existente — autenticación con Auth.js v5, base de datos PostgreSQL gestionada con Prisma, uploads a Cloudflare R2 y un sistema de notificaciones con campanita ya funcionando — cubre la mayor parte de los cimientos, de modo que el único componente nuevo de infraestructura es un servicio de mensajería en tiempo real.

\$0/mes

costo fijo inicial (todo en free tiers)

15–18 días

de trabajo efectivo en 8 fases

100

usuarios conectados a la vez
soportados gratis

La recomendación técnica central es usar **Pusher Channels** en su plan gratuito Sandbox como capa de entrega instantánea de mensajes, manteniendo PostgreSQL como única fuente de verdad. Cada mensaje se persiste primero en la base de datos y luego se emite un evento por REST hacia los canales privados de la conversación; si el servicio de eventos llegara a fallar, el chat degrada a una actualización por recarga sin perder ningún mensaje. Las notas de voz y las imágenes reutilizan el flujo de subida presignada a R2 que el proyecto ya usa para las fotos de los negocios, con un nuevo tipo de archivo y autorización ajustada.

La viabilidad está sujeta a tres condiciones que conviene nombrar desde el inicio. Primera: el plan gratuito de Pusher admite hasta 100 conexiones simultáneas, margen holgado para la escala actual del directorio pero que debe monitorearse desde el panel del proveedor, con la alternativa de migrar a Ably (200 conexiones gratis) sin rehacer el esquema. Segunda: la moderación no es opcional en un producto social — el plan incluye reportes, bloqueos y una pestaña nueva en el panel de administración existente desde el día uno. Tercera: el proyecto hoy no tiene middleware global ni Redis, por lo que el endpoint de envío necesita rate limiting por usuario y validación estricta con zod, siguiendo los patrones que el propio código ya estableció.

El resto del documento detalla el estado verificado de la plataforma (capítulo 3), el reto central del tiempo real sobre Vercel y las alternativas comparadas (capítulo 4), la arquitectura propuesta con su diagrama (capítulo 5), el modelo de datos y el alcance funcional de la versión 1 (capítulos 6 y 7), el plan de fases con estimaciones (capítulo 8), los costos y límites del plan gratuito (capítulo 9), los riesgos con sus mitigaciones (capítulo 10) y, por último, la alternativa sin terceros y las decisiones que el dueño debe confirmar antes de arrancar (capítulo 11).

2. Contexto, objetivo y decisiones

CONECTA-LT es hoy el directorio digital de la vida nocturna de Los Teques: locales, promociones, eventos, reseñas y un planner que ya reúnen a una audiencia con intereses en común. Esa audiencia, sin embargo, no tiene forma de coordinarse dentro de la plataforma: quien quiere preguntar por un plan del viernes o dividir una reserva termina saliéndose hacia redes sociales o llamando por teléfono. Un chat entre usuarios convierte el directorio en comunidad, aumenta el tiempo de permanencia en el sitio y genera una razón más para crear una cuenta y mantener la sesión iniciada. Es importante subrayar que este canal convive con el WhatsApp del dueño (+58 422-0117206), que sigue siendo la vía de atención directa del equipo; el chat nuevo es persona-a-persona, no un reemplazo del soporte.

Decisiones confirmadas por el dueño

Pregunta	Decisión	Consecuencia directa
¿Quiénes se comunican?	Usuarios entre sí	No se construye mensajería usuario-negocio en la v1; las fichas no cambian
¿Qué nivel de "en vivo"?	Tiempo real verdadero	Requiere servicio de mensajería externo compatible con Vercel (capítulo 4)
¿Alcance de la versión 1?	Completo: 1-a-1, grupos y notas de voz	Plan por fases más largo; el MVP 1-a-1 queda publicable en la fase 3

Tabla 1: las tres decisiones de producto ya tomadas.

Las preguntas restantes del cuestionario no obtuvieron respuesta, así que el plan adopta los valores recomendados por defecto que se listan a continuación. Ninguno de ellos condiciona la arquitectura: cambiarlos después solo afecta configuración o ajustes menores de interfaz, no obliga a rehacer el modelo de datos ni los endpoints. Por eso el documento los marca explícitamente como ajustables y los recoge de nuevo en la checklist final del capítulo 11, donde el dueño puede confirmarlos o modificarlos en un solo gesto antes de arrancar.

Valores por defecto aplicados (ajustables)

Dimensión	Valor adoptado	Si el dueño lo cambia
Moderación	Básica: reportar y bloquear, revisión en panel admin	El nivel "completo" añade filtro de palabras y suspensiones (+1–2 días)
Identidad en el chat	Perfil real: nombre y avatar de la cuenta	Un alias editable es un campo nuevo y una casilla de privacidad

Dimensión	Valor adoptado	Si el dueño lo cambia
Notificaciones	Badge de no leídos en la web (sin correos)	Añadir email usa RESEND_API_KEY, ya definida en el proyecto (+0,5 día)
Presupuesto	Solo free tiers	Con hasta \$20/mes se eliminan los límites del capítulo 9
Formato del entregable	Este documento PDF	La implementación puede arrancar por separado cuando se apruebe

Tabla 2: decisiones por defecto que el dueño puede ajustar sin costo de re-arquitectura.

3. Estado actual de la plataforma (evidencia verificada)

Antes de estimar esfuerzo se auditó el código real del repositorio (rama main, estado del 25 de septiembre de 2026). La conclusión es que CONECTA-LT parte con una base notablemente sólida para un chat: identidad, persistencia, archivos y panel de administración ya existen y siguen patrones claros que el chat puede reutilizar en lugar de inventar. En paralelo, quedó confirmado que no existe ninguna infraestructura de tiempo real en el proyecto — ni WebSockets, ni SSE, ni servicios de mensajería — y que la producción corre como funciones serverless de Vercel, condición que define el diseño del capítulo 4.

Lo que ya existe y se reutiliza

Capacidad	Dónde vive hoy	Cómo la usa el chat
Autenticación y roles	Auth.js v5 con JWT; helpers <code>getCurrentUser/requireUser</code> ; roles <code>USER...ADMIN</code>	Cada endpoint del chat exige sesión; moderación usa <code>requireRole</code>
Notificaciones y campanita	<code>notificationService.notify()</code> + <code>NotificationsBell</code> con badge en el navbar	Aviso "nuevo mensaje" y el patrón visual del badge de no leídos
Subida de archivos a R2	<code>POST /api/upload/presign</code> (SDK S3), proxy <code>/api/images</code> , CORS configurado	Notas de voz e imágenes con un nuevo tipo CHAT y <code>auth requireUser</code>
Actualización periódica	React Query con <code>refetchInterval</code> de 30 s (campanita, reservas, dueños)	Refresco del contador de no leídos y de la bandeja como respaldo
Panel de administración	Vista admin con pestañas (<code>AdsTab</code> , <code>EventsTab...</code>) y <code>guards requireRole</code>	Nueva pestaña "Chat" para reportes y mensajes eliminados
Validación y limitación	<code>zod</code> disponible (usado en planner); <code>rate limiter</code> artesanal en memoria	Esquemas de entrada del chat y límite de envíos por usuario por minuto

Capacidad	Dónde vive hoy	Cómo la usa el chat
Lenguaje visual	glass-card, tema obsidiana/oro, Dialog de shadcn, framer-motion	La interfaz del chat calca los mismos componentes y se integra al SPA

Tabla 3: capacidades existentes verificadas en el código y su reuso directo.

Lo que hay que construir desde cero

Pieza faltante	Alcance
Modelos de chat en Prisma	Conversation, Participant y Message (más reportes y bloqueos); migración y índices
Canal de tiempo real	Cuenta y claves de Pusher, cliente pusher-js, endpoint de autorización de canales
API de mensajería	Endpoints de bandeja, mensajes paginados, marcado de lectura y no leídos
Interfaz del chat	Vista "chat" del SPA: bandeja de conversaciones, ventana de conversación, badge
Endpoint de subida para chat	Extensión del presign actual: tipos CHAT_AUDIO/CHAT_IMAGE con requireUser
Moderación de chat	Reportes, bloqueos, soft delete y pestaña admin correspondiente

Tabla 4: piezas ausentes confirmadas; ninguna depende de decisiones externas.

El diagnóstico cierra con un matiz técnico importante: el patrón vigente de la plataforma es el sondeo cada 30 segundos con React Query, y ese patrón es perfecto para una campanita o un tablero de dueño, pero no entrega la sensación de conversación instantánea que el dueño pidió. La producción en Vercel funciona con funciones efímeras, así que la instantaneidad exige un servicio dedicado de entrega de eventos. Esa es exactamente la brecha que el siguiente capítulo analiza y que la arquitectura propuesta resuelve con la incorporación de un único proveedor externo.

4. El reto clave: tiempo real sobre Vercel serverless

Un chat "en vivo real" necesita que el servidor pueda avisar al navegador en el momento en que llega un mensaje, en lugar de esperar a que el cliente pregunte. Eso se logra tradicionalmente con una conexión persistente (WebSocket) mantenida por un proceso de servidor siempre encendido. El despliegue de CONECTA-LT, en cambio, usa funciones serverless: cada petición levanta un proceso efímero que vive lo que dura la respuesta y desaparece. No hay proceso persistente donde "colgar" conexiones, y por eso un servidor de WebSockets propio dentro de Next.js simplemente no funciona en producción, aunque sí en el entorno local de desarrollo. Ésta es la única decisión realmente estructural del proyecto y conviene tomarla con todas las opciones sobre la mesa.

Alternativas para la entrega instantánea

Opción	Gratis incluye	Integración con Vercel	Riesgo principal
Pusher Channels (recomendada)	100 conexiones simultáneas; 200k mensajes/día	Nativa: la API dispara eventos por REST desde las routes; cliente pusher-js en el navegador	Límite del plan gratuito; lock-in mitigable con una capa propia
Ably	200 conexiones simultáneas; 6M mensajes/mes	Igual de natural (SDK REST + cliente), muy similar a Pusher	Mismo tipo de límite; cuotas menos difundidas
Supabase Realtime	200 conexiones; 2M mensajes/mes	Requiere BD en Supabase o replicación: hoy la BD es Neon, no encaja sin migrar	Mover o duplicar la base de datos solo por el chat
SSE sobre Vercel	Sin servicio externo	Las conexiones largas se cortan en funciones serverless y no hay pub/sub interno	Frágil y con reconexiones constantes; no recomendado
WebSocket propio (Railway/Fly)	Servidor propio siempre encendido (~\$5/mes)	Otro despliegue que mantener, TLS, monitoreo y despliegue separado del web	Costo fijo + carga operativa; contradice el presupuesto \$0

Tabla 5: comparación de las cinco alternativas evaluadas para entrega instantánea.

Decisión propuesta: Pusher Channels (plan Sandbox, gratis). Se integra de forma natural con las API routes de Next.js, cobra por conexión simultánea — el recurso que de verdad escala un chat — y permite encapsular su SDK detrás de una interfaz propia, de modo que migrar después a Ably (o a un plan pago) no toca el modelo de datos ni la interfaz de usuario. Si el dueño prefiriera cero dependencias externas, el capítulo 11 describe el plan B "casi en vivo" con sondeo de 3–5 segundos que reutiliza el patrón React Query existente.

Conviene precisar qué hace cada pieza en la opción elegida. El navegador se suscribe a canales privados cuyo nombre incluye el identificador de la conversación; la autorización de esa suscripción la emite el propio backend, que verifica con la sesión de Auth.js que quien pide escuchar el canal es participante de esa conversación. Los eventos que viajan por el canal son avisos ligeros (identificador del mensaje nuevo, estado de escritura, confirmaciones de lectura), nunca el historial completo: ante cualquier duda el cliente vuelve a preguntar a la API, que es la autoridad. Así el sistema permanece correcto aunque el canal de eventos se pierda unos segundos, y la información sensible de los mensajes no circula por un tercero más de lo necesario.

5. Arquitectura propuesta

La arquitectura mantiene toda la lógica de negocio en el proyecto actual y añade exactamente una pieza externa: el bus de eventos de Pusher. Los navegadores hablan con las API routes de Vercel por HTTPS (con sesión Auth.js verificada en cada llamada), las routes persisten todo en PostgreSQL mediante Prisma y, tras persistir, disparan el evento en vivo hacia los canales privados. Los archivos pesados — notas de voz e imágenes — no pasan por el backend: el navegador sube directo a R2 con una URL presignada de corta vida, igual que ya hace hoy con las fotos de los negocios, y el mensaje final guarda solo la referencia. La figura 1 de esta sección resume las tres capas y el recorrido de un mensaje.

El recorrido de un mensaje

Cuando el usuario A pulsa enviar, el navegador hace un POST a la API con el cuerpo validado por zod y la sesión ya verificada por requireUser. La API comprueba que A es participante de la conversación, persiste el mensaje en PostgreSQL, actualiza la marca de último mensaje de la conversación y, solo si todo eso tuvo éxito, dispara por REST el evento "nuevo mensaje" hacia el canal privado de esa conversación. El navegador del usuario B — que estaba suscrito desde que abrió la conversación — recibe el aviso al instante, pide los datos del mensaje por su identificador y lo muestra, con la conversación ya ordenada y los contadores de no leídos actualizados. En el caso inverso, el receptor confirmó lectura con una llamada de marcado que actualiza Participant.lastReadAt, y esa confirmación también viaja por el canal para que el emisor vea los "vistos" sin recargar.

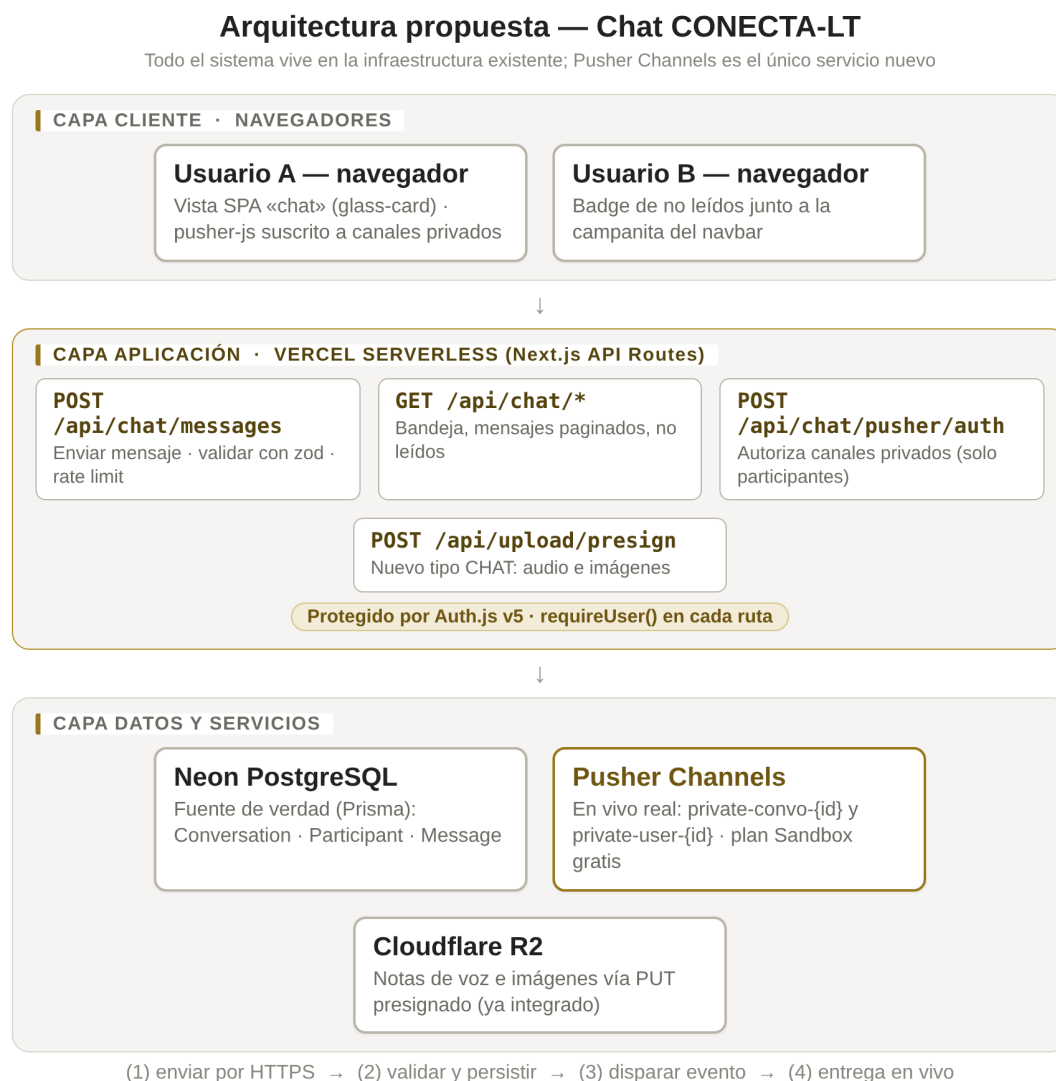


Figura 1: arquitectura en tres capas; Pusher es la única pieza nueva de infraestructura.

Canales privados y seguridad de la suscripción

Se usan dos familias de canales, ambas privadas. El canal por conversación (private-convo-{id}) transporta mensajes, indicador de "escribiendo..." y confirmaciones de lectura de esa conversación concreta. El canal por usuario (private-user-{id}) transporta avisos globales — "te escribió alguien con quien aún no tienes conversación abierta" — para refrescar el badge del navbar aunque la bandeja esté cerrada. La autorización de ambas suscripciones pasa por un endpoint propio (/api/chat/pusher/auth) que consulta la sesión y la tabla de participantes antes de firmar; nadie puede escuchar un canal sin ser participante. Los payloads de los eventos son deliberadamente mínimos: identificadores y contadores, nunca contenido completo, de modo que el contenido sensible solo circula entre navegador y API por HTTPS.

Por qué PostgreSQL sigue siendo la fuente de verdad

El diseño sigue la regla de oro de los sistemas de mensajería sobre infraestructura serverless: persistir primero, notificar después. Si el evento de Pusher se pierde — por una caída puntual del proveedor o por una desconexión del receptor — nada se corrompe: el mensaje ya está en la base de datos y cualquier cliente lo recupera al reabrir la conversación o al reconectar, porque la bandeja y el historial se leen siempre de la API y no del canal. Esta separación convierte al proveedor de tiempo real en una capa prescindible y reemplazable, lo que reduce el riesgo de dependencia a un problema de disponibilidad pasajera y no de pérdida de datos.

6. Modelo de datos

El esquema añade cinco tablas que siguen las convenciones ya establecidas en el proyecto: identificadores `cuid()`, fechas de creación y actualización automáticas, e índices en toda clave foránea consultada. El corazón son tres tablas — `Conversation`, `Participant` y `Message` — más dos de soporte para la moderación (`ChatReport` y `BlockedUser`). El diseño soporta sin cambios tanto conversaciones de dos personas como grupos: la diferencia queda expresada en el campo `type` y en la cantidad de participantes, no en tablas distintas.

```
model Conversation {
  id          String   @id @default(cuid())
  type        String   @default("DIRECT") // DIRECT | GROUP
  name        String?
  createdById String
  lastMessageAt DateTime @default(now())
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  participants Participant[]
  messages    Message[]
  @@index([lastMessageAt])
}
```

```
model Participant {
  id          String   @id @default(cuid())
  conversationId String
  userId      String
  role        String   @default("MEMBER") // OWNER | MEMBER
  lastReadAt  DateTime @default(now())    // base del contador de no leídos
  joinedAt    DateTime @default(now())
  conversation Conversation @relation(fields: [conversationId], references: [id])
  user        User        @relation(fields: [userId], references: [id])
  @@unique([conversationId, userId])
  @@index([userId])
}
```

```

model Message {
  id          String      @id @default(cuid())
  conversationId String
  senderId    String
  type        String      @default("TEXT")    // TEXT | IMAGE | AUDIO | SYSTEM
  body        String?      // texto (max 2000, validado con zod)
  mediaUrl    String?      // /api/images/chat/... (R2)
  mediaKey    String?
  durationMs  Int?         // duracion de nota de voz
  createdAt   DateTime     @default(now())
  deletedAt   DateTime?    // soft delete por moderacion
  @@index([conversationId, createdAt])
}

```

Contador de no leídos y reutilización de notificaciones

El contador de no leídos se calcula con `Participant.lastReadAt`: para cada conversación se cuenta cuántos mensajes hay con fecha posterior a esa marca, y el índice (`conversationId`, `createdAt`) vuelve esa consulta barata incluso con historiales largos. Cuando el usuario abre la conversación, el cliente marca lectura y el emisor recibe la confirmación por el canal. Para la campanita global del navbar se reutiliza la infraestructura existente de `Notification` y `notificationService.notify()`: un mensaje nuevo genera una notificación del tipo correspondiente, con lo cual el badge hereda todo el comportamiento que ya tienen las reservas y las reseñas hoy, sin duplicar lógica.

Moderación y retención

La eliminación de mensajes es siempre un soft delete: el campo `deletedAt` marca el contenido como retirado y la interfaz lo muestra como "mensaje eliminado", pero los metadatos permanecen para que los administradores puedan investigar reportes. `ChatReport` registra el mensaje denunciado, el denunciante y el estado del caso, y `BlockedUser` impide que dos cuentas vuelvan a abrir conversaciones entre sí. Sobre las notas de voz en R2 se propone una política de retención de 90 días con limpieza programada, que mantiene el almacenamiento dentro del nivel gratuito sin que el dueño tenga que tocar nada; la decisión final se recoge en la checklist del capítulo 11.

7. Alcance funcional de la versión 1

Esta versión se define por lo que el usuario ve y puede hacer, no por la tecnología: abrir una conversación con otra persona, escribir en vivo, crear un grupo para coordinar una salida, mandar una nota de voz o una imagen, enterarse de que le escribieron aunque esté navegando en otra vista, y contar con herramientas básicas para reportar o bloquear conductas molestas. Las siguientes subsecciones detallan cada comportamiento y señalan qué patrón existente reutiliza cada uno, que es la razón por la que el alcance completo cabe en el plan de fases del capítulo 8.

Bandeja y ventana de conversación

El chat entra como una vista más del SPA existente (view "chat" en el store de Zustand), accesible desde un icono nuevo en el navbar junto a la campanita. La bandeja lista las conversaciones ordenadas por último mensaje con su previsualización y contador de no leídos; al entrar, la ventana muestra el historial paginado con carga incremental hacia atrás y mantiene el scroll anclado al último mensaje. Todo se construye con los componentes visuales que la plataforma ya usa — glass-card sobre el tema obsidiana/oro, Dialog de shaden para nueva conversación, animaciones discretas con framer-motion — de modo que el chat parezca parte del producto y no un módulo pegado. En móvil ocupa toda la pantalla, siguiendo el comportamiento responsive que las demás vistas ya resuelven.

Entrega en vivo, escritura y confirmación de lectura

Los mensajes enviados aparecen de inmediato en la conversación con su estado "enviando" (optimista) y confirman en cuanto la API responde; si la llamada falla, el mensaje se marca con error y ofrece reintentar, sin duplicar al reintentar porque la API es idempotente por identificador de cliente. El indicador "escribiendo..." se emite por el canal de la conversación con una cadencia limitada para no saturar eventos, y las confirmaciones de lectura (Participant.lastReadAt) se propagan igual, mostrando a cada emisor qué mensajes fueron vistos. Como respaldo permanente, React Query refresca la bandeja y los contadores cada 30 segundos — el mismo intervalo que ya usa la campanita — de modo que una caída del canal en vivo degrada la experiencia sin romperla.

Grupos

Cualquier usuario puede crear un grupo con nombre e imagen, invitar participantes desde una búsqueda por nombre y salir cuando quiera. El creador queda como OWNER y puede invitar o retirar miembros; para la v1 se mantiene esa distinción simple de dos roles, evitando la matriz de permisos fina que encarece este tipo de productos. Los eventos administrativos — "Carla agregó a Luis", "Luis salió del grupo" — se registran como mensajes de tipo SYSTEM, que la interfaz renderiza centrados y tenues, igual que los sistemas de mensajería convencionales. La entrega en vivo de grupos usa exactamente el mismo canal por conversación, por lo que no hay lógica de distribución adicional que mantener.

Notas de voz

El botón de micrófono usa la API MediaRecorder del navegador, presente en todos los navegadores modernos, para grabar en formato webm/opus con un límite de 60 segundos y 2 MB por clip. Al terminar, el navegador pide una URL presignada al endpoint ampliado y sube el audio directo a R2, y el mensaje resultante guarda la referencia con su duración para renderizar una burbuja con reproducción y duración visible. El único ajuste en el backend es añadir el tipo CHAT_AUDIO con sus MIME permitidos y cambiar la autorización de ese caso de dueños de negocio a requireUser, porque el presign actual está restringido a fotos de fichas. Los navegadores sin permiso de micrófono simplemente no muestran el botón, y el chat funciona con texto e imágenes sin pérdida alguna.

Imágenes en el chat

Las imágenes siguen el mismo cauce que las notas de voz: presign con tipos jpeg, png y webp, límite de 5 MB y subida directa a R2 con la compresión del lado del cliente que ya aplica el componente ImageUploadZone cuando el archivo excede el tamaño. La diferencia con el flujo actual es solo la autorización (usuario cualquiera con sesión) y la clave de almacenamiento (chat/ en lugar de businesses/), de modo que el proxy /api/images existente sirve el contenido sin cambios. Las burbujas de imagen muestran una miniatura que abre el visor en tamaño completo, reutilizando el patrón de galería que la ficha de cada local ya usa.

Badge de no leídos en el navbar

El navbar gana un icono de mensajería con contador agregado de no leídos, visible solo con sesión iniciada, al lado de la campanita de notificaciones. El contador se refresca en vivo gracias al canal privado por usuario y, como respaldo, con el sondeo de 30 segundos; al abrir una conversación, su contador individual baja a cero y el agregado se recalcula. Los usuarios sin sesión no ven el icono, y la AgeGate existente sigue protegiendo el acceso igual que con el resto de la aplicación. Esta pieza es deliberadamente conservadora: hereda los patrones de use-notifications-sync para minimizar el código nuevo y el riesgo visual en el navbar.

Moderación básica

Cada mensaje ofrece en su menú contextual las acciones "reportar" y "bloquear usuario" (además de "eliminar" para el propio emisor). Un reporte crea un ChatReport en cola con el mensaje completo adjunto; un bloqueo cierra la vía de conversación futura entre ambas cuentas sin eliminar el historial visible para el bloqueante. En el panel de administración aparece la pestaña "Chat" con las colas de reportes y la posibilidad de retirar contenido (soft delete) o suspender a un usuario reincidente, siguiendo al dedillo el patrón de componentes y guards de las pestañas AdsTab y EventsTab existentes. Con esto el dueño cumple la promesa mínima de un espacio social moderado sin invertir todavía en filtrado automático de palabras.

8. Plan de implementación por fases

El plan divide el trabajo en ocho fases ordenadas para que, al final de cada una, exista algo demostrable en el entorno de preview — nunca un gran estreno al final. Las fases 0 a 3 conforman el MVP publicable: con unas 8 a 10 jornadas de trabajo efectivo, dos usuarios ya pueden chatear en vivo desde la producción con 1-a-1, badge y notificaciones. Las fases 4 a 7 suman el alcance completo que el dueño eligió para la v1 (grupos, notas de voz, moderación y endurecimiento). Las duraciones asumen sesiones de trabajo concentradas y revisión del dueño al cierre de cada fase.

Fases, entregables y duración estimada

Fase	Entregable demostrable	Trabajo principal	Duración
F0 · Preparación	Credenciales y esquema listos en preview	Cuenta Pusher, variables PUSHER_* en Vercel, migración Prisma de las 5 tablas	0,5–1 d
F1 · API de mensajería	Chat funciona recargando la página	Endpoints de bandeja, mensajes paginados, lectura y no leídos; validación zod	2–3 d
F2 · Tiempo real	Mensajes aparecen al instante en dos navegadores	Cliente pusher-js, endpoint de autorización de canales, eventos, UI optimista	2 d
F3 · Interfaz completa	MVP usable end-to-end en producción	Vista chat, bandeja, ventana responsive, badge en navbar, icono de menú	3–4 d
F4 · Grupos	Grupo creado y coordinado desde la web	Crear/invitar/salir, roles OWNER/MEMBER, mensajes SYSTEM	2–3 d
F5 · Voz e imágenes	Nota de voz e imagen enviadas desde el móvil	Presign CHAT, grabador MediaRecorder, reproductor, compresión de imágenes	2–3 d
F6 · Moderación y avisos	Reporte gestionado desde el panel admin	Reportes, bloqueos, soft delete, pestaña admin "Chat", email opcional	2 d
F7 · Cierre y QA	QA firmado y límites activos	Rate limit por usuario, límites de tamaño, QA multi-dispositivo, accesibilidad	2 d

Tabla 6: plan de fases; el total asume revisión del dueño al cierre de cada fase.

Dos consideraciones de gestión completan el plan. Primero, la migración de la fase 0 es aditiva — crea tablas nuevas sin tocar las existentes — por lo que el riesgo sobre lo ya funcionando es mínimo y reversible. Segundo, el orden de las fases 4 a 6 es intercambiable: si el dueño prefiere ver notas de voz antes que grupos, basta reordenar sin costo, porque cada fase depende solo del MVP (fases 0 a 3) y no de sus hermanas.

9. Costos y límites del plan gratuito

El escenario inicial no agrega costo fijo mensual: cada pieza del chat corre dentro de un nivel gratuito que la plataforma ya usa o que se crea nuevo. El gasto real del proyecto sigue siendo cero mientras la audiencia se mantenga en la escala actual del directorio, y la tabla siguiente deja explícito qué servicio se agota primero y qué hacer cuando eso ocurra. La regla de decisión es sencilla: Pusher es el primer recurso que se satura, porque cobra por conexiones simultáneas; todo lo demás crece por uso y tiene margen holgado.

Costo mensual por servicio en el escenario inicial

Servicio	Plan	Costo	Qué cubre para el chat
Vercel	Hobby (actual)	\$0	Invocaciones de las API routes del chat
Neon PostgreSQL	Free (actual)	\$0	Almacenamiento y consultas de mensajes
Pusher Channels	Sandbox (nuevo)	\$0	Entrega en vivo: 100 conexiones simultáneas y 200k mensajes/día
Cloudflare R2	Free (actual)	\$0	10 GB de notas de voz e imágenes (millones de operaciones incluidas)
Total		\$0/mes	

Tabla 7: escenario inicial; ningún pago fijo nuevo.

Cuándo se rompe cada nivel gratuito y qué hacer

Límite	Umbral	Señal temprana	Plan de acción
Conexiones simultáneas de Pusher	100 usuarios conectados a la vez	Panel de Pusher con picos sobre 80	Migrar a Ably (200 gratis) vía la capa propia, o Pusher Starter (\$49/mes)
Mensajes por día (Pusher)	200k eventos/día	Panel de Pusher cerca del cupo	Es un techo muy alto; si se alcanza, ya se justificó el plan pago
Almacenamiento R2	10 GB de audio/imagen	Métrica de bucket sobre 8 GB	Aplicar retención de audios a 90 días (F6) y compresión de imágenes
Invocaciones Vercel / compute Neon	Cuotas del nivel actual	Sin cambio relevante: el chat añade poco tráfico HTTP	Escalar plan cuando el resto de la plataforma lo exija

Tabla 8: umbrales de saturación y respuestas previstas.

Para dimensionar el margen: 100 conexiones simultáneas significa cien personas con la web abierta y el chat activo en el mismo instante, no cien usuarios al día. Un directorio local como CONECTA-LT opera hoy muy por debajo de ese piso en horas pico, y el panel del proveedor permite ver la curva real desde el primer día, de modo que la decisión de pagar llegue con datos y no con sustos. Si el producto creciera hasta agotar el nivel gratuito, el costo de siguiente escalón (unos \$49/mes en Pusher, o \$0 migrando a Ably) entra en la conversación con la evidencia de uso delante.

10. Riesgos, seguridad y moderación

Ningún riesgo del proyecto es estructural, pero conviene tratarlos explícitamente porque un chat introduce conductas sociales que el directorio no tenía. La tabla siguiente ordena los seis riesgos principales con su probabilidad, impacto y mitigación prevista; después se detallan las garantías de seguridad que sostienen todo el diseño. El hilo conductor es que el backend nunca confía en el cliente: cada lectura y cada envío re-verifican sesión y pertenencia a la conversación en el servidor.

Matriz de riesgos y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
Se superan las 100 conexiones simultáneas del plan gratis	Media	Alto	Monitoreo en panel desde F2; capa propia de transporte para migrar a Ably (200 gratis) o plan pago
Spam o acoso entre usuarios	Media	Alto	Rate limit por usuario (patrón del planner), límites de longitud, reportes y bloqueo desde F6
Contenido inapropiado en conversaciones	Media	Alto	Moderación día 1, soft delete con auditoría, pestaña admin, nota expresa en Términos
Caída del proveedor de tiempo real	Baja	Medio	Postgres es fuente de verdad: el chat degrada a refresco por sondeo sin perder mensajes
Complejidad desbordada por permisos de grupos	Media	Medio	Solo dos roles (OWNER/MEMBER) en v1; permisos finos quedan para una v2 si hay demanda
Dependencia de un tercero (lock-in)	Baja	Medio	SDK encapsulado en interfaz propia; modelo de datos y UI no conocen al proveedor

Tabla 9: los seis riesgos identificados, ordenados por necesidad de atención.

Garantías de seguridad del diseño

Cuatro garantías sostienen la seguridad del chat. Primera: autorización en cada movimiento, con `requireUser` en cada endpoint y verificación de participación antes de leer o escribir cualquier conversación, replicando el scoping por usuario que hoy aplica el endpoint de notificaciones. Segunda: canales privados firmados por el backend, de modo que la suscripción a `private-convo` o `private-user` pasa por el endpoint de autorización y no existe forma de escuchar conversaciones ajenas manipulando el cliente. Tercera: validación y contención, donde zod valida todo cuerpo entrante (tipos, longitudes, formatos) y el rate limiting por usuario contiene el abuso automatizado, ampliando el patrón que el planner ya demostró. Cuarta: privacidad por defecto, porque los mensajes solo existen entre cuentas autenticadas, los payloads de eventos transportan identificadores y no contenido, y los Términos del servicio incorporan una sección que explica qué se guarda, quién puede verlo y cómo se elimina.

11. Alternativa sin terceros y decisiones pendientes

Si el dueño prefiriera no incorporar ningún proveedor externo, existe un plan B completamente viable en su escala actual: el modo "casi en vivo". Consiste en sondear la conversación abierta cada 3 a 5 segundos con React Query — el mismo mecanismo que la campanita ya usa cada 30 — y sondear la bandeja con un intervalo mayor. El resultado se siente como una conversación con medio segundo a cinco segundos de retardo, funciona al 100% dentro de Vercel y Neon, cuesta cero dólares y no agrega dependencias. Sus dos concesiones son la instantaneidad imperfecta y un consumo mayor de invocaciones cuando muchos usuarios tienen conversaciones abiertas; para la escala actual ambas son asumibles.

El atractivo del plan B es que no es un callejón sin salida: el modelo de datos, la API, la interfaz y la moderación son idénticos a los del plan recomendado — solo cambia el transporte de avisos. Incorporar Pusher más adelante equivale a añadir el cliente y el endpoint de autorización, sin migraciones ni reescrituras. Por eso la decisión entre ambos modos puede posponerse hasta la fase 2 sin costo: se construye primero todo lo común (fases 0 y 1) y se elige el transporte cuando el chat sea visible en preview. La recomendación técnica sigue siendo Pusher para cumplir el "en vivo real" que el dueño eligió, pero la arquitectura no queda rehén de esa elección.

Checklist de decisiones del dueño antes de arrancar

Cinco confirmaciones desbloquean la fase 0 y ninguna requiere trabajo técnico del dueño. Responderlas en una sola sesión basta para agendar el inicio; cualquiera que cambie respecto de este documento se incorpora como una edición menor del plan, no como un rediseño.

- **1. Cuenta de Pusher:** crear la cuenta gratuita y autorizar la carga de las variables `PUSHER_APP_KEY`, `PUSHER_APP_SECRET`, `PUSHER_APP_ID` y `PUSHER_CLUSTER` en Vercel (dos minutos de registro; el resto lo hace el equipo).
- **2. Alcance de moderación inicial:** confirmar el nivel básico (reportes, bloqueos, pestaña admin) o pedir el nivel completo con filtro de palabras y suspensiones (+1–2 días).
- **3. Retención de notas de voz:** aprobar los 90 días propuestos para la limpieza automática en R2, o indicar otra política.
- **4. Correos opcionales:** decidir si los mensajes llegan también por email al usuario desconectado usando `RESEND_API_KEY`, ya definida en el proyecto (+0,5 día de trabajo).
- **5. Nombre visible del producto:** confirmar "Mensajes" como etiqueta del icono en el navbar y del título de la vista, o proponer otro nombre al gusto.

Con esas cinco confirmaciones, el siguiente paso natural es ejecutar la fase 0 y la fase 1 en una sesión de trabajo, dejando el chat consultable en preview aunque sea recargando la página. A partir de ahí, cada fase añade una capacidad visible y el dueño puede decidir en cualquier punto si publicar el MVP antes de completar el alcance total, con la meta de que la gente de Los Teques encuentre en conectalt.com una plataforma que también sirve para quedar, no solo para elegir dónde.